# Confidence-Aware Graph Convolutional Networks for Social Bot Detection: A Message-Passing Reweighting Approach

Kanalbay Kassymkhan
School of Software Engineering
Astana IT University
Astana, Kazakhstan
255479@astanait.edu.kz

***Abstract*—Social bots distort engagement metrics, amplify disinformation, and erode trust in online discourse, which motivates graph-based detectors that exploit relational structure. Standard Graph Convolutional Networks (GCNs) aggregate neighbor messages with weights that depend only on node degree, so a neighbor with a confident, well-separated prediction contributes exactly as much as one with a noisy, borderline prediction. We propose a Confidence-Aware GCN that reweights message passing by the model's own predictive confidence in each neighbor, computed as the maximum softmax probability from the previous epoch and renormalized over each node's incoming edges. The mechanism adds no learnable parameters and is implemented as a custom PyTorch Geometric layer. We verify it on a synthetic social graph (500 nodes) and evaluate it on a labeled Twitter bot dataset (5 000-account subsample, cosine-similarity $k$-NN graph) against MLP, GCN, GraphSAGE and GAT baselines with 5-fold cross-validation repeated over 5 seeds, paired Wilcoxon tests with Holm correction, threshold-independent metrics, a threshold-matched comparison and ten ablations. The precision gain seen in an earlier single-split experiment (+3.0 points) does not survive this protocol: Conf-GCN matches GCN in precision (0.732 vs. 0.731, $p = 0.83$) but loses 0.9 F1 points and 2.5 AUROC points, and at matched recall GCN is slightly *more* precise. Uniform, shuffled and inverted confidences reproduce the full model to within one F1 point, showing that the effect comes from the change of normalization and the missing self-loop, not from confidence; on real data the confidence weights deviate from mean aggregation by only 6.5% on average. Graph construction matters far more: on the description-only graph (edge homophily 0.62) every GCN-type model is beaten by a feature-only MLP (F1 0.807), whereas a $k$-NN graph over descriptions and metadata (homophily 0.76) lifts all GNNs above the MLP (F1 0.814–0.825). We report these negative and positive findings, release all code, and discuss when confidence weighting could still be useful.**

***Index Terms*—bot detection, graph convolutional network, message passing, confidence weighting, node classification, negative results**

## I. Introduction

Social media platforms increasingly contend with coordinated inauthentic accounts, commonly called social bots, that inflate follower counts, manufacture trending topics, and propagate disinformation at scale [6]. Bots frequently operate in coordinated clusters and use templated, repetitive language, so both the relational structure of a social graph and the semantic content of posts carry complementary signal. This has motivated a shift from per-account classifiers toward Graph Neural Networks (GNNs), which propagate information along follow, mention, reply, or content-similarity edges and can therefore detect coordinated behavior invisible to a single-account feature vector.

A standard GCN [1] aggregates each node's neighborhood with degree-normalized weights that are agnostic to how reliable a neighbor's representation currently is. In bot detection this is a potential weakness: when bots are deliberately disguised to resemble genuine users, a small number of cross-class edges connect bot clusters to the human subgraph, and an ambiguous account near the decision boundary then contributes to its neighbors' updates with the same weight as an account classified with near-certainty. Noisy, low-confidence signal can thus leak across the class boundary.

This paper studies a Confidence-Aware GCN that addresses this weakness at the message-passing level. Each neighbor's contribution is scaled by the model's own confidence in that neighbor's class prediction from the previous epoch, renormalized so incoming weights sum to one. The mechanism requires no additional learnable parameters. In an earlier single-split experiment [25] the method raised precision by 3.0 points at a 7.6-point recall cost. Here we test whether that trade-off is a genuine effect of the confidence signal.

**Research questions. RQ1**: does the mechanism behave correctly (convergence; equivalence to uniform aggregation when all confidences are equal)? **RQ2**: how does it change accuracy, precision, recall, F1, AUROC and AUC-PR relative to GCN, GraphSAGE, GAT and a feature-only model, and is the change statistically reliable? **RQ3**: is the effect attributable to the confidence signal itself, rather than to the change in normalization or to a shift of the decision threshold? **RQ4**: how much does the construction of the similarity graph matter compared with the aggregation rule?

**Contributions.** (1) A parameter-free confidence-weighted message-passing layer, implemented as a custom PyTorch Geometric `MessagePassing` module, with unit tests for its formal properties. (2) A correctness validation on a synthetic social graph with known ground truth. (3) An evaluation on a real labeled Twitter dataset with repeated cross-validation,

TABLE I
POSITIONING RELATIVE TO RELATED AGGREGATION SCHEMES

| Method | Weight source | Extra params | Uses pred. confidence |
|---|---|---|---|
| GCN [1] | degree | no | no |
| GAT [2] | learned attention | yes | no |
| GraphSAGE [3] | mean/pool | yes | no |
| ConfGCN [12] | learned label confidence | yes | indirectly |
| Ours | softmax confidence | no | yes |

paired significance tests, stronger GNN baselines, a threshold-matched comparison and ten ablations that separate the confidence signal from its confounders. (4) A negative result: under this protocol the confidence signal has no measurable benefit, and we explain why (bounded, near-uniform weights). (5) A positive finding: graph construction decides whether any GNN beats a feature-only model on this tabular benchmark. Code, configurations and per-run results are public.[1]

## II. Related Work

### A. Feature-based bot detection

Early detectors relied on hand-engineered behavioral and metadata features (posting frequency, account age, follower-to-friend ratios) combined with supervised classifiers; Botometer (formerly BotOrNot) blends a large number of such features in a supervised ensemble [4]. Such approaches are effective against unsophisticated bots but degrade as bots adopt human-like patterns, and generalize poorly across datasets [5], [6]. Feature selection studies on Twitter bot detection show that profile and description features alone reach moderate accuracy [11].

### B. Graph-based bot detection

Graph-based approaches exploit the observation that bots, even when individually convincing, tend to be structurally distinguishable: they cluster densely, share near-duplicate content, or connect to common amplification targets. GCNs [1] and inductive GraphSAGE-style models [3] have been applied to spam and bot detection [7]. BotRGCN and the Relational Graph Transformer extend this to heterogeneous graphs with multiple relation types [8], [10], evaluated on the TwiBot-20 benchmark [9]. Graph Attention Networks (GATs) learn per-edge attention weights instead of fixing them to degree-based normalization [2]. The benefit of message passing depends strongly on homophily, i.e. the fraction of edges joining same-class nodes [23].

### C. Confidence- and uncertainty-aware aggregation

Several lines of work use model confidence. Pseudo-labeling and self-training use confident predictions on unlabeled data as additional supervision [13]. The closest graph-learning work is ConfGCN [12], which estimates a per-node confidence over label distributions and uses it to modulate neighbor influence in semi-supervised node classification. Our approach differs in the signal and its cost: ConfGCN maintains a learned, per-node label-distribution estimate with additional parameters and a distribution-matching objective, whereas we derive confidence directly from the classifier's own softmax output, with no extra parameters, and study it specifically for bot detection. The reliability of softmax confidence itself is known to be imperfect and can be improved by calibration such as temperature scaling [14]. Stochastic regularizers on message passing, such as DropEdge [15], also aim to reduce the effect of noisy neighbors, but do so randomly and without regard to prediction reliability.

### D. Similarity graphs for tabular data

Sentence-BERT embeddings [16] enable similarity-based edges between accounts posting near-duplicate content even without explicit social links. For tabular bot datasets that provide rich per-account metadata but no follow graph, a $k$-nearest-neighbor graph over embeddings is a common substitute for explicit structure, a construction long used in graph-based semi-supervised learning [17].

## III. Problem Formulation

Let $G = (V, E)$ be a social graph, where $V$ is the set of accounts and $E$ the set of follow, mention, reply, or content-similarity edges. Each node $v$ has a feature vector $x_v \in \mathbb{R}^d$ built from a Sentence-BERT embedding of the account text concatenated with normalized profile metadata. A label $y_v \in \{0, 1\}$ (0 = human, 1 = bot) is known for a subset of nodes. The task is transductive semi-supervised node classification: learn $f : G \to \{0, 1\}^{|V|}$ predicting the label of every node, including those withheld from training, by propagating information along $E$.

## IV. Methodology

### A. Baseline GCN

The baseline stacks three `GCNConv` layers (PyTorch Geometric) mapping $d \to 64 \to 64 \to 2$, with ReLU and dropout ($p = 0.5$). Node $i$ is updated at layer $l$ as

$$h_i^{(l+1)} = \sigma\Big( \sum_{j \in \mathcal{N}(i) \cup \{i\}} \frac{1}{\sqrt{\hat{d}_i \hat{d}_j}} W^{(l)} h_j^{(l)} \Big), \tag{1}$$

where $\hat{d}_i$ is the degree of $i$ including a self-loop. Training uses cross-entropy with Adam [19].

### B. Confidence-Aware GCN

The confidence of node $j$ is the maximum softmax probability of its logits $z_j$ from the previous epoch:

$$c_j = \max_k \ \mathrm{softmax}(z_j)_k. \tag{2}$$

For $C = 2$ classes, $c_j \in [0.5, 1]$. For every directed edge $j \to i$ the weight is renormalized over the incoming edges of $i$:

$$\tilde{w}_{j \to i} = \frac{c_j^{\alpha}}{\sum_{k \in \mathcal{N}(i)} c_k^{\alpha}}, \tag{3}$$

[1] https://github.com/solodancer235-star/confidence-aware-gcn-bot-detection

TABLE II
CONFIGURATION OF BASELINE AND CONFIDENCE-AWARE MODELS

| Component | Baseline | Conf.-Aware |
|---|---|---|
| Conv. layer | `GCNConv` (PyG) | `ConfWeightedConv`* |
| Layers | 3 | 3 |
| Hidden dim. | 64 | 64 |
| Dropout | 0.5 | 0.5 |
| Learnable params (Twitter) | 29 570 | 29 570 |
| Optimizer | Adam (selected on val.) | Adam (selected on val.) |

*Custom `MessagePassing` layer; mean-style normalization, no self-loop.

with $\alpha = 1$ in the proposed model, and the node update becomes

$$h_i^{(l+1)} = \sigma\Big( \sum_{j \in \mathcal{N}(i)} \tilde{w}_{j \to i}\, W^{(l)} h_j^{(l)} \Big). \tag{4}$$

Confidence is initialized to 1 for every node. At each epoch a forward pass produces logits for the loss and optimizer step; a second no-gradient forward pass in evaluation mode then recomputes the confidence vector for the next epoch. Confidence is stored as a plain tensor and updated inside a no-gradient context, so gradients never flow through it; this avoids a feedback loop between the loss and a quantity that depends on it. Confidence is computed from model outputs only and uses no labels, so evaluation nodes are not leaked.

### C. Properties and limits of the mechanism

**Reduction to uniform aggregation.** When $c_j = 1$ for all $j$ (the first forward pass), Eq. (4) is mean aggregation over $\mathcal{N}(i)$. This is not identical to the symmetric degree normalization with self-loops used by the baseline (Eq. 1). The two models therefore differ in normalization and self-loop handling in addition to the confidence signal; we address this explicitly with ablations (Section VI-E) rather than assuming the difference is attributable to confidence alone. The layer is also invariant to a global rescaling of the confidences, since only ratios enter Eq. (3).

**Bounded influence.** Because $c_j \in [0.5, 1]$ in the binary case, the weights of any two neighbors of the same node differ by at most a factor of $2^{\alpha}$. With $\alpha = 1$ the mechanism can therefore only moderately rebalance a neighborhood; we measure how much it actually does in Section VII-G.

### D. Architectural parity

Both models use three layers, hidden dimension 64, dropout 0.5 and the same hyper-parameter search (Table II); they have exactly the same number of learnable parameters (verified by a unit test). Parity of capacity does not imply that confidence is the only difference, so we complement it with the ablations below.

## V. IMPLEMENTATION

The system is implemented in Python with PyTorch and PyTorch Geometric [20], sentence-transformers (`all-MiniLM-L6-v2`, 384-dimensional output) [16], NetworkX for graph construction, scikit-learn for $k$-NN graphs, scaling and metrics, and SciPy for statistical tests.

`ConfidenceWeightedConv` extends `MessagePassing` with additive aggregation. Its `forward` method takes a per-node source-weight vector as an argument, computes per-edge weights as the source node's weight divided by the sum of weights over the target node's incoming edges (scatter-add indexed by target), and its `message` method multiplies each transformed source representation by this weight. `ConfidenceAwareGCN` stacks three such layers, stores confidence as a plain module attribute (not a registered parameter), lazily initializes it to ones on the first forward call, and exposes a no-gradient `update_confidence` method that recomputes it as the row-wise maximum softmax probability of the latest logits. A `mode` switch implements the ablation controls (uniform, shuffled and inverted confidence) and a `self_loop` flag adds self-loops inside the confidence-weighted layer. Unit tests check that incoming weights sum to one, that $c \equiv 1$ reproduces mean aggregation exactly, scale invariance, parameter parity with `GCNConv`, and that confidence carries no gradient.

## VI. EXPERIMENTAL SETUP

### A. Synthetic social graph

To verify correctness under controlled conditions we generate a graph of 500 nodes: 400 humans connected by a Barabási–Albert preferential-attachment model ($m = 3$), and 100 bots in two dense, fully connected clusters of 50, plus 20 random bot–human edges simulating disguised bots (3 661 structural edges). Human text is drawn from 15 topical templates, bot text from 10 near-duplicate cryptocurrency-spam templates with minor lexical variation. Each node's 384-dimensional embedding is its feature vector, and edges are added between node pairs whose cosine similarity exceeds 0.85 (6 663 new undirected edges, 10 324 in total).

### B. Real-world Twitter dataset

We use the publicly available Twitter human/bot dataset [21] (37 438 accounts, 25 013 humans, 12 425 bots), with labels obtained through the Botometer API combined with manual verification and 20 metadata columns. We work with a cleaned public redistribution of it [22] that keeps the 29 407 accounts whose interface language is English or unset (19 676 humans, 9 731 bots; class ratio 2.02:1 vs. 2.01:1 in the original) and strips URLs, @-mentions, e-mail addresses and emoji from the descriptions. Because labels partly derive from a feature-based detector, label noise correlated with metadata features is possible (Section VIII-D).

For computational reasons on CPU, we draw a stratified subsample of 5 000 accounts (2 500 humans, 2 500 bots; fixed seed); 1 516 of them (30%) have an empty description. Node features are 394-dimensional: a 384-d embedding of the profile description, six numeric fields (status, follower, friend and favorite counts, account age in days, average tweets per day) log-transformed and standardized with training-node statistics of each fold, and four binary fields (default profile, default profile image, verified, geo-enabled).

**Graph construction.** As the dataset has no explicit follow graph, the default graph is a symmetrized cosine $k$-NN graph ($k = 10$) over the description embeddings [17] (84 462 directed edges, edge homophily 0.617). To answer RQ4 we also build a *feature graph*: a $k$-NN graph ($k = 10$) over the description embedding concatenated with the z-scored ten metadata fields, the metadata block scaled by $1/\sqrt{10}$ so that both blocks have comparable norm (77 696 edges, homophily 0.756). Both constructions are label-free.

### C. Compared models

All GNNs use three layers, hidden dimension 64 and dropout 0.5:

- **MLP**: same features, no graph (isolates the value of structure).
- **GCN** [1]: baseline.
- **GraphSAGE** [3]: mean aggregator with a separate root (self) transformation.
- **GAT** [2]: learned attention; total hidden width 64 split over $h$ heads.
- **Conf-GCN** (ours).

### D. Evaluation protocol

**Splits and repetition.** Each model is evaluated with stratified 5-fold cross-validation repeated over 5 random seeds (25 runs per model), with the training portion of each fold further split to obtain a validation set (60/20/20 overall). All models see identical splits.

**Hyper-parameter selection.** Learning rate $\in \{0.01, 0.005\}$, weight decay $\in \{5\times10^{-4}, 5\times10^{-5}\}$ and, for GAT, the number of heads $\in \{1, 4, 8\}$ are selected by mean validation F1 over the five folds of seed 0 only; the test folds are never used. Selected: MLP, GraphSAGE and GAT ($h = 1$) $\mathrm{lr} = 0.005$, $\mathrm{wd} = 5 \times 10^{-4}$; GCN and Conf-GCN $\mathrm{lr} = 0.01$, $\mathrm{wd} = 5 \times 10^{-5}$. All ablations reuse the Conf-GCN setting. The epoch count is fixed at 100 for all models, without early stopping.

**Metrics.** Accuracy, precision, recall and F1 for the bot class at the default threshold 0.5, plus the threshold-independent AUROC and AUC-PR (average precision), and the expected calibration error (ECE, 10 bins).

**Statistical testing.** We report mean $\pm$ standard deviation over the 25 runs and compare Conf-GCN against each baseline with a paired two-sided Wilcoxon signed-rank test over matched runs, with Holm correction across the comparisons of a table [18], [24]. Because the runs share data across folds and seeds, $p$-values are treated as indicative rather than exact.

**Threshold-matched analysis.** A precision gain with a recall loss could simply reflect a different operating point on the same ranking. For every run we therefore (i) take the validation recall of Conf-GCN at threshold 0.5 as a target, (ii) choose each baseline's threshold on *its validation set* as the largest threshold reaching that recall, and (iii) compare test precision at these thresholds. We also report full precision–recall curves.

TABLE III
SYNTHETIC GRAPH, SINGLE STRATIFIED 60/20/20 SPLIT

| Metric | Baseline | Conf.-Aware | Δ |
|---|---|---|---|
| Accuracy | 1.0000 | 1.0000 | 0.0000 |
| Precision | 1.0000 | 1.0000 | 0.0000 |
| Recall | 1.0000 | 1.0000 | 0.0000 |
| F1-score | 1.0000 | 1.0000 | 0.0000 |

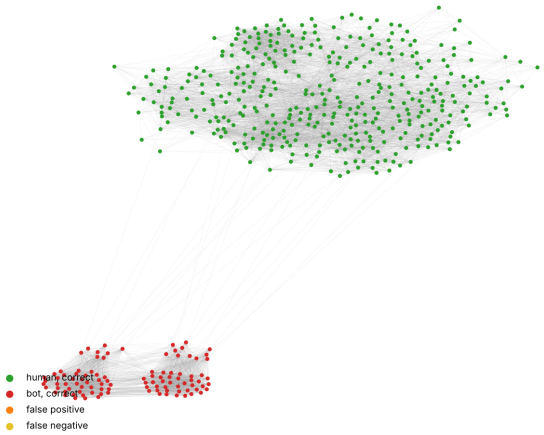



Fig. 1. Synthetic graph (500 nodes, 10 324 edges) colored by the Conf-GCN prediction: correctly classified humans (green) form a loose preferential-attachment community; correctly classified bots (red) form two dense, well-separated clusters. No node is misclassified.

### E. Ablations

To attribute the effect to the confidence signal itself, we run, with all other settings fixed:

1) **Uniform-confidence control**: same layer with $c_j \equiv 1$ (mean aggregation, no self-loop). This separates the effect of confidence from the change in normalization.
2) **Shuffled confidence**: confidences randomly permuted across nodes each epoch (same marginal distribution, no node-specific information).
3) **Inverted weighting**: weights proportional to $1-c_j+0.5$, favoring low-confidence neighbors.
4) **Exponent** $\alpha$: $\alpha \in \{0.5, 2, 4\}$ ($\alpha = 0$ is the uniform control, $\alpha = 1$ the full model).
5) **Self-loop**: self-loops added to the confidence-weighted layer, with real and with uniform confidence.
6) **Graph density**: $k \in \{5, 10, 20\}$ in the $k$-NN graph, for GCN and Conf-GCN.
7) **Graph construction**: description graph vs. feature graph for all GNNs.

## VII. RESULTS

### A. Synthetic graph (RQ1)

Both models reach the task ceiling: accuracy, precision, recall and F1 are all 1.0000 for the baseline and the confidence-aware model (Table III, Fig. 1); the same holds for MLP, GraphSAGE and GAT and for all 25 cross-validation runs

TABLE IV
REAL TWITTER DATA, SINGLE SPLIT, AS REPORTED IN OUR EARLIER STUDY [25]

| Metric | Baseline | Conf.-Aware | Δ |
|---|---|---|---|
| Accuracy | 0.7150 | 0.7130 | −0.0020 |
| Precision | 0.7002 | 0.7300 | +0.0298 |
| Recall | 0.7520 | 0.6760 | −0.0760 |
| F1-score | 0.7252 | 0.7020 | −0.0232 |

of every model (125 runs, every metric 1.000). This is expected from the data-generating process: the bot clusters are fully connected subgraphs whose members share near-identical embeddings, producing a dense web of semantic edges within each cluster and almost none to the human subgraph, so the classes are separable by structure and by features alone. The task therefore verifies correctness (RQ1) but cannot discriminate between the models. Training loss falls to below $10^{-3}$ ($9.8 \times 10^{-4}$ for GCN, $6.5 \times 10^{-4}$ for Conf-GCN) for both models, and the mean confidence of the confidence-aware model rises from 0.998 at epoch 20 to 0.999 at epoch 100 (Fig. 3), showing that the mechanism converges as intended. The unit tests confirm the formal properties of Section IV-C, in particular exact equivalence to mean aggregation when $c \equiv 1$.

### B. Preliminary single-split result

Table IV reproduces the original single train/test split on a 5 000-account subsample of the full 37 438-account release [25]. The confidence-aware model improved precision by 2.98 points while recall fell by 7.60 points and F1 by 2.32 points; accuracy was essentially unchanged. On a single split, differences of this size cannot be separated from sampling noise, which is why the protocol of Section VI-D was added. Absolute numbers below are not directly comparable with Table IV because the English-only redistribution yields a different subsample.

### C. Extended comparison (RQ2)

Table V shows the cross-validated comparison. Three findings stand out. First, the precision gain of Table IV does not replicate: Conf-GCN and GCN have the same mean precision (0.732 vs. 0.731, $p = 0.83$). The recall loss does replicate in direction (−2.0 points), so Conf-GCN has a lower F1 than GCN (0.731 vs. 0.740, $p = 0.004$) and is statistically indistinguishable from GAT in F1. Second, the threshold-independent metrics are clearly worse for Conf-GCN: AUROC 0.794 vs. 0.819 for GCN and GAT, AUC-PR 0.772 vs. 0.824, i.e. its ranking of accounts is worse, not merely shifted. Third, and most importantly for interpreting any GNN result on this benchmark, the feature-only MLP is the best model (F1 0.807, AUROC 0.890), tied with GraphSAGE (F1 0.803), whose separate root transformation lets it fall back on a node's own features. Models that mix a node's representation with its description neighbors (GCN, GAT, Conf-GCN) lose 6.7–7.6 F1 points relative to ignoring the graph. The run-to-run standard deviation of F1 (0.016–0.019 for GCN-type models) is of the same order as the single-split differences in Table IV, which explains why a single split was misleading.

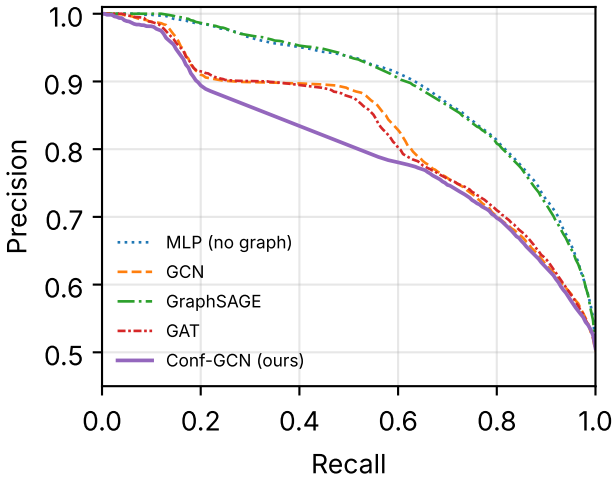



Fig. 2. Precision–recall curves on the test folds (precision interpolated on a common recall grid and averaged over the 25 runs). Description graph, $k = 10$.

### D. Ablations (RQ3)

Table VI isolates the confidence signal. The uniform-confidence control, which keeps the layer but discards confidence, is *at least as good* as the full model (F1 0.736 vs. 0.731, AUROC 0.795 vs. 0.794). Shuffled confidence, which preserves the marginal distribution but destroys node identity, and inverted weighting, which favors the *least* confident neighbors, both land within 0.5 F1 points of the full model and are not significantly different from it. Sharpening the weights does not help either: $\alpha = 2$ and $\alpha = 4$ slightly lower F1 (0.728, 0.727) and AUROC (0.793, 0.788). The behavior of Conf-GCN is therefore explained by its normalization, not by confidence.

The self-loop variants identify which part of the normalization matters. Adding a self-loop to the uniform layer recovers GCN-level performance (F1 0.742, AUROC 0.817 vs. GCN 0.740, 0.819); adding it to the confidence-weighted layer recovers the AUROC (0.815) but not the F1 (0.734). The 2.5-point AUROC gap between Conf-GCN and GCN in Table V is thus almost entirely due to the missing self-loop, i.e. to the node's own features being diluted by its neighbors. We answer RQ3 negatively: on this benchmark the confidence signal itself has no measurable benefit, and the precision-oriented behavior suggested by the single split is not a property of the method.

### E. Threshold-matched precision

Table VII asks whether a practitioner who wants Conf-GCN's operating point could obtain it more cheaply by moving the threshold of a baseline. At matched recall (0.730–0.733 for all models) GCN is 0.9 points and GAT 1.2 points *more* precise than Conf-GCN, and MLP and GraphSAGE are about

TABLE V

REAL TWITTER DATA (DESCRIPTION $k$-NN GRAPH, $k = 10$), 5-FOLD CV $\times$ 5 SEEDS (MEAN $\pm$ STD OVER 25 RUNS). $p$: PAIRED WILCOXON VS. CONF-GCN, HOLM-CORRECTED OVER THE FOUR BASELINES.

| Model | Acc. | Prec. | Rec. | F1 | AUROC | AUC-PR | $p$ (F1) | $p$ (Prec.) |
|---|---|---|---|---|---|---|---|---|
| MLP (no graph) | 0.807±0.008 | 0.805±0.012 | 0.809±0.023 | 0.807±0.010 | 0.890±0.008 | 0.894±0.009 | <0.001 | <0.001 |
| GCN | 0.737±0.014 | 0.731±0.020 | 0.752±0.037 | 0.740±0.016 | 0.819±0.013 | 0.824±0.014 | 0.004 | 0.833 |
| GraphSAGE | 0.803±0.011 | 0.805±0.014 | 0.801±0.030 | 0.803±0.014 | 0.887±0.010 | 0.893±0.009 | <0.001 | <0.001 |
| GAT | 0.737±0.014 | 0.744±0.021 | 0.726±0.045 | 0.734±0.020 | 0.819±0.013 | 0.823±0.013 | 0.230 | 0.003 |
| Conf-GCN (ours) | 0.731±0.016 | 0.732±0.023 | 0.732±0.042 | 0.731±0.019 | 0.794±0.013 | 0.772±0.013 | — | — |

TABLE VI

ABLATIONS, DESCRIPTION GRAPH (MEAN $\pm$ STD OVER 25 RUNS). $p$: PAIRED WILCOXON OF F1 VS. FULL CONF-GCN, HOLM-CORRECTED OVER THE NINE COMPARISONS.

| Variant | Prec. | Rec. | F1 | AUROC | $p$ (F1) |
|---|---|---|---|---|---|
| GCN (reference) | 0.731±0.020 | 0.752±0.037 | 0.740±0.016 | 0.819±0.013 | 0.015 |
| Uniform confidence ($c \equiv 1$) | 0.730±0.021 | 0.745±0.039 | 0.736±0.018 | 0.795±0.013 | 0.026 |
| Shuffled confidence | 0.729±0.024 | 0.741±0.033 | 0.735±0.017 | 0.795±0.013 | 0.104 |
| Inverted weighting | 0.736±0.017 | 0.731±0.036 | 0.733±0.020 | 0.797±0.013 | 0.668 |
| $\alpha = 0.5$ | 0.730±0.025 | 0.739±0.039 | 0.734±0.016 | 0.795±0.013 | 0.668 |
| $\alpha = 2$ | 0.733±0.028 | 0.725±0.042 | 0.728±0.017 | 0.793±0.014 | 0.668 |
| $\alpha = 4$ | 0.725±0.026 | 0.733±0.044 | 0.727±0.017 | 0.788±0.014 | 0.668 |
| Uniform conf. + self-loop | 0.729±0.023 | 0.756±0.034 | 0.742±0.016 | 0.817±0.013 | <0.001 |
| Conf-GCN + self-loop | 0.730±0.028 | 0.741±0.051 | 0.734±0.018 | 0.815±0.013 | 0.668 |
| Conf-GCN (full, $\alpha = 1$) | 0.732±0.023 | 0.732±0.042 | 0.731±0.019 | 0.794±0.013 | — |

TABLE VII

TEST PRECISION OF EACH BASELINE AT A THRESHOLD TUNED ON ITS VALIDATION SET TO CONF-GCN'S VALIDATION RECALL (MEAN $\pm$ STD OVER 25 RUNS). $\Delta$ = CONF-GCN $-$ BASELINE; $p$: PAIRED WILCOXON, HOLM-CORRECTED.

| Baseline | Rec. (base) | Prec. (base) | Prec. (Conf) | $\Delta$ | $p$ |
|---|---|---|---|---|---|
| MLP (no graph) | 0.733±0.038 | 0.850±0.021 | 0.732±0.023 | -0.118 | <0.001 |
| GCN | 0.730±0.038 | 0.740±0.022 | 0.732±0.023 | -0.009 | <0.001 |
| GraphSAGE | 0.731±0.035 | 0.848±0.022 | 0.732±0.023 | -0.116 | <0.001 |
| GAT | 0.731±0.039 | 0.743±0.022 | 0.732±0.023 | -0.012 | <0.001 |

TABLE VIII

EFFECT OF GRAPH CONSTRUCTION ($k = 10$, MEAN $\pm$ STD OVER 25 RUNS). DESCRIPTION GRAPH: HOMOPHILY 0.617; FEATURE GRAPH: 0.756.

| | Description graph | | Feature graph | |
|---|---|---|---|---|
| Model | F1 | AUROC | F1 | AUROC |
| GCN | 0.740±0.016 | 0.819±0.013 | 0.820±0.012 | 0.905±0.006 |
| GraphSAGE | 0.803±0.014 | 0.887±0.010 | 0.825±0.011 | 0.906±0.007 |
| GAT | 0.734±0.020 | 0.819±0.013 | 0.814±0.010 | 0.898±0.006 |
| Conf-GCN (ours) | 0.731±0.019 | 0.794±0.013 | 0.818±0.011 | 0.901±0.006 |
| MLP (no graph) | F1 0.807±0.010, AUROC 0.890±0.008 | | | |

12 points more precise; all differences are consistent across runs ($p < 0.001$). The precision–recall curves (Fig. 2) show the same picture at every operating point: the Conf-GCN curve lies below GCN and GAT, which lie well below MLP and GraphSAGE. Any precision-oriented operating point is therefore better obtained by raising the threshold of a baseline.

### F. Graph construction and density (RQ4)

The description graph is a weak structure: only 61.7% of its edges join accounts of the same class (50% would be random for a balanced sample), and the 30% of accounts with an empty description share an identical embedding, so their neighborhoods are essentially arbitrary among themselves. Building the $k$-NN graph over descriptions *and* metadata raises edge homophily to 75.6% and changes the conclusions (Table VIII): every GNN gains 2–9 F1 points ($p < 10^{-6}$ for each), and all of them now beat the feature-only MLP (GraphSAGE 0.825, GCN 0.820, Conf-GCN 0.818, GAT 0.814 vs. 0.807; $p \leq 0.001$ for GCN, GraphSAGE and Conf-GCN, $p = 0.001$ for GAT, unadjusted). On this better graph Conf-GCN is statistically tied with GCN in F1 (0.818 vs. 0.820, $p = 0.12$) and precision (0.831 vs. 0.838, $p = 0.08$), with a small but consistent AUROC deficit (0.901 vs. 0.905, $p < 0.001$); confidence weighting does not become beneficial when the graph is more homophilous.

Graph density matters much less than graph construction. On the description graph, GCN reaches F1 0.732/0.740/0.734 for $k = 5/10/20$ and Conf-GCN 0.729/0.731/0.726; GCN is ahead of Conf-GCN at every $k$ (unadjusted $p = 0.15$, 0.002, $< 0.001$).

### G. Why the confidence signal has no effect

We trained Conf-GCN on the five folds of seed 0 and inspected the final confidence vector and the resulting edge weights. Averaged over the 25 main runs, the mean confidence on the real data rises only from 0.586 after the first epoch to 0.749 at epoch 100 (Fig. 3), in contrast to the synthetic graph where it saturates near 0.999; the 10th and 90th percentiles of the final confidences are 0.572 and 0.903. Inside a neighborhood the largest weight is a median 1.45$\times$ the smallest (95th percentile 1.82$\times$), and the weights deviate from uniform $1/\deg(i)$ by only 6.5% on average. The bounded-influence property of Section IV-C therefore binds in practice: the layer is very close to mean aggregation, which is why the uniform control reproduces it. The confidence signal is also only moderately informative: as a score for whether a test node is classified correctly it reaches an AUROC of only 0.66 (0.77 on the feature graph, where the weights still deviate from uniform by just 7.1%), and the softmax outputs of Conf-GCN are not better calibrated than those of the baselines (ECE 0.041 vs. 0.034 for GCN and 0.033 for MLP). Sharpening the weights with $\alpha > 1$ amplifies this partly wrong signal and lowers AUROC (Table VI). The cost of the mechanism is a second forward pass per epoch; on a shared two-core CPU a

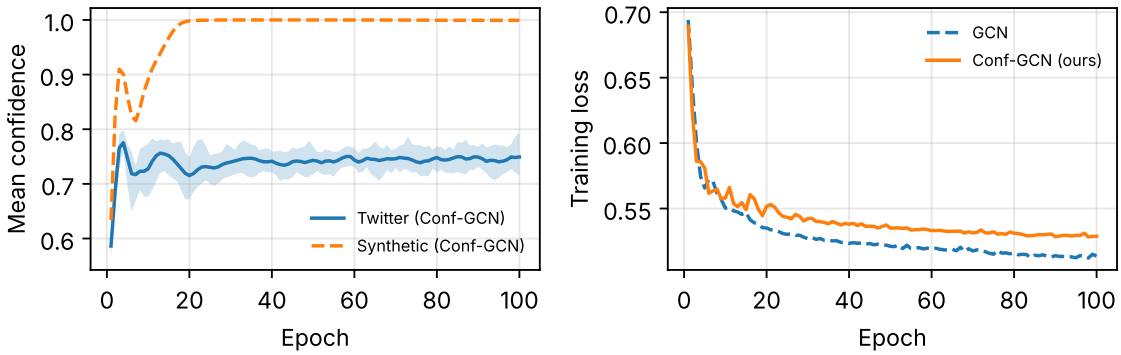



Fig. 3. Left: mean confidence of Conf-GCN over training (real data: mean and min–max range over 25 runs; synthetic: single run). Right: mean training loss of GCN and Conf-GCN on the real data.

Conf-GCN run (100 epochs) took 16.9 s on average vs. 14.3 s for GCN.

## VIII. Discussion

### A. Answers to the research questions

**RQ1:** the layer is correct: it converges on the synthetic graph, reduces exactly to mean aggregation for equal confidences, and has the same parameter count as GCN. **RQ2:** on the real data Conf-GCN does not improve any metric over GCN; it is equal in precision and worse in recall, F1, AUROC and AUC-PR, and all GCN-type models are beaten by an MLP on the description graph. **RQ3:** the differences from GCN are explained by the change of normalization (mean aggregation without self-loop), not by confidence: uniform, shuffled and inverted confidences behave like the full model, and a threshold-tuned GCN dominates it at matched recall. **RQ4:** graph construction dominates the aggregation rule; a more homophilous feature graph lifts every GNN above the MLP, but it does not make confidence weighting useful.

### B. Precision–recall trade-off and practical significance

In moderation pipelines false positives are often more costly than false negatives: incorrectly labeling a genuine user as a bot can trigger restrictions and complaints, whereas a missed bot may be caught on a later pass as more interaction data accumulates. Under such an asymmetric cost structure a precision gain can be valuable even at the cost of recall. This argument was the motivation for the method, but it requires that the precision gain cannot be obtained more cheaply. Our threshold-matched analysis shows that it can: raising the threshold of GCN, or better of GraphSAGE or the MLP, gives higher precision at the same recall. The precision–recall trade-off observed in Table IV is therefore best interpreted as split noise around a different operating point, and we withdraw the "implicit regularizer" interpretation proposed in our earlier study.

### C. Absolute accuracy

The best models reach about 81–83% accuracy on balanced data using profile metadata and description text only, without tweet history or an explicit follow graph. Reported accuracy for comparable feature sets varies widely across datasets and protocols [5], [11], so we avoid claiming a specific literature band and instead rely on the in-paper MLP, GCN, GraphSAGE and GAT comparisons.

### D. Threats to validity

**Graph construction:** the $k$-NN graphs are derived from the same embeddings (and, for the feature graph, the same metadata) used as features, so edges encode similarity already present in the features; real follow or mention graphs may behave differently. **Labels:** partly derived from Botometer, which is itself a feature-based detector, so metadata-based models may be favored. **Data:** we used an English-only cleaned redistribution and a 5 000-account subsample; the code also accepts the original release. **Statistics:** runs share data across folds and seeds, so Wilcoxon $p$-values are indicative. **Hyperparameters:** the grid is small and was selected on seed 0; all ablations reuse the Conf-GCN setting. **Bounded weights:** the binary confidence range restricts how far the mechanism can rebalance a neighborhood; multi-class problems or calibrated confidences may behave differently. **Confidence reliability:** softmax confidence is not guaranteed to be calibrated [14].

## IX. Conclusion

We presented a Confidence-Aware GCN that reweights neighbor contributions during message passing by each neighbor's own predictive confidence, recomputed every epoch and renormalized, with no additional learnable parameters. The mechanism is correct and cheap, but a rigorous evaluation on a real Twitter bot dataset shows that it does not deliver the precision gain suggested by an earlier single-split experiment: it ties GCN in precision, loses in recall, F1 and AUROC, and is dominated by a threshold-tuned GCN at matched recall. Controls with uniform, shuffled and inverted confidence reproduce its behavior, and diagnostics show why: in the binary case the confidence weights stay within a few percent of mean aggregation. The decisive factor on this benchmark is instead the graph: with a description-only $k$-NN graph no

GCN-type model beats a feature-only MLP, while a $k$-NN graph over descriptions and metadata lifts all GNNs above it. We hope these results, together with the released code and per-run data, help others avoid over-interpreting single-split gains of aggregation tweaks and draw attention to graph construction for tabular bot datasets.

## X. Future Work

Several directions remain open. Because the weights are bounded, a stronger signal is needed for confidence to matter: a learnable exponent $\alpha$ ($w_{j\to i} \propto c_j^{\alpha}$), which recovers uniform aggregation at $\alpha = 0$, or a margin- or entropy-based confidence after temperature scaling [14]. Confidence could modulate learned attention in GAT [2] instead of replacing degree normalization, or be added on top of GCN's symmetric normalization with self-loops so that only the confidence signal differs from the baseline. Evaluation on multi-relational graphs with explicit follow, mention, reply and retweet edges such as TwiBot-20 [9], and on heterogeneous architectures such as BotRGCN [8] with per-relation confidence, would show whether the mechanism helps when the graph is real rather than similarity-derived. Finally, the link between confidence-weighted aggregation and pseudo-labeling suggests a self-training extension in which high-confidence predictions seed supervision on unlabeled nodes [13].

## Code and Data Availability

All code, configurations, per-run results (CSV), predicted probabilities and the scripts that generate every table and figure in this paper are available at https://github.com/solodancer235-star/confidence-aware-gcn-bot-detection. The full protocol runs on a two-core CPU in about two hours.


## Acknowledgment

The author thanks the School of Software Engineering at Astana IT University for guidance throughout the design and evaluation of this work.


## References


[1] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *Proc. ICLR*, 2017.

[2] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *Proc. ICLR*, 2018.

[3] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proc. NeurIPS*, 2017.

[4] C. A. Davis, O. Varol, E. Ferrara, A. Flammini, and F. Menczer, "BotOrNot: A system to evaluate social bots," in *Proc. 25th Int. Conf. Companion on World Wide Web*, 2016, pp. 273–274.

[5] K.-C. Yang, O. Varol, P.-M. Hui, and F. Menczer, "Scalable and generalizable social bot detection through data selection," in *Proc. AAAI*, vol. 34, no. 1, 2020, pp. 1096–1103.

[6] S. Cresci, "A decade of social bot detection," *Commun. ACM*, vol. 63, no. 10, pp. 72–83, 2020.

[7] S. Ali Alhosseini, R. Bin Tareaf, P. Najafi, and C. Meinel, "Detect me if you can: Spam bot detection using inductive representation learning," in *Companion Proc. World Wide Web Conf.*, 2019, pp. 148–153.

[8] S. Feng, H. Wan, N. Wang, and M. Luo, "BotRGCN: Twitter bot detection with relational graph convolutional networks," in *Proc. IEEE/ACM ASONAM*, 2021.

[9] S. Feng, H. Wan, N. Wang, J. Li, and M. Luo, "TwiBot-20: A comprehensive Twitter bot detection benchmark," in *Proc. 30th ACM CIKM*, 2021, pp. 4485–4494.

[10] S. Feng, Z. Tan, R. Li, and M. Luo, "Heterogeneity-aware Twitter bot detection with relational graph transformers," in *Proc. AAAI*, vol. 36, no. 4, 2022, pp. 3977–3985.

[11] D. Lee, S. Lee, and H. Yu, "Discriminative feature selection for multi-class Twitter bot detection," *IEEE Access*, vol. 9, pp. 32752–32762, 2021.

[12] S. Vashishth, P. Yadav, M. Bhandari, and P. Talukdar, "Confidence-based graph convolutional networks for semi-supervised learning," in *Proc. AISTATS*, 2019.

[13] D.-H. Lee, "Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks," in *ICML Workshop on Challenges in Representation Learning*, 2013.

[14] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. ICML*, 2017, pp. 1321–1330.

[15] Y. Rong, W. Huang, T. Xu, and J. Huang, "DropEdge: Towards deep graph convolutional networks on node classification," in *Proc. ICLR*, 2020.

[16] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proc. EMNLP*, 2019, pp. 3982–3992.

[17] X. Zhu, Z. Ghahramani, and J. Lafferty, "Semi-supervised learning using Gaussian fields and harmonic functions," in *Proc. ICML*, 2003, pp. 912–919.

[18] J. Demšar, "Statistical comparisons of classifiers over multiple data sets," *J. Mach. Learn. Res.*, vol. 7, pp. 1–30, 2006.

[19] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," in *Proc. ICLR*, 2015.

[20] M. Fey and J. E. Lenssen, "Fast graph representation learning with PyTorch Geometric," in *ICLR Workshop on Representation Learning on Graphs and Manifolds*, 2019.

[21] D. M. Gutiérrez, "Twitter human bots dataset," Kaggle, 2021. [Online]. Available: https://www.kaggle.com/datasets/davidmartngutirrez/twitter-bots-accounts

[22] Bot-Ani project, "Processed Twitter human/bot dataset (English accounts)," GitHub, 2024. [Online]. Available: https://github.com/Max-SF1/Bot-Ani-RWKV-twitter-bot-detection

[23] J. Zhu, Y. Yan, L. Zhao, M. Heimann, L. Akoglu, and D. Koutra, "Beyond homophily in graph neural networks: Current limitations and effective designs," in *Proc. NeurIPS*, 2020.

[24] S. Holm, "A simple sequentially rejective multiple test procedure," *Scand. J. Statist.*, vol. 6, no. 2, pp. 65–70, 1979.

[25] K. Kassymkhan, "Confidence-aware graph convolutional networks for social bot detection: A message-passing reweighting approach," in *Proc. 2026 IEEE 3rd Int. Student Conf. on Digital Generation*, Astana, Kazakhstan, 2026.